
CS771 Major Assignment 1, 2

Team oneSHOT

Amitesh Patra (251110010)
Gaurav Singh Bora (251110031)
K Yashwant Rao (251110039)
Manas Dhaketa (251110046)
Manoj Kumar Mali (251110048)
Major Sankalp Sharma (251110063)

Problem 1.1: Semi-Parametric Regression Using Custom Kernels

Question 1: Derivation of Semi-Parametric Kernel $\tilde{K}$

We are given data pairs $(x, \mathbf{z})$, where:

- $x \in \mathbb{R}$ is a scalar (representing video length).
- $\mathbf{z} \in \mathbb{R}^D$ is a vector (representing popularity and difficulty).

The number of staff members required, y , is modeled as:

$$y = w(\mathbf{z}) \cdot x + b \quad (1)$$

Here, the coefficient $w(\mathbf{z})$ depends non-linearly on $\mathbf{z}$. To capture this, we assume a semi-parametric form:

$$w(\mathbf{z}) = \mathbf{p}^\top \phi(\mathbf{z})$$

where $\phi(\cdot)$ is the feature map associated with the polynomial kernel $K_z(\mathbf{z}_1, \mathbf{z}_2) = (\mathbf{z}_1^\top \mathbf{z}_2 + c)^d$.

Our goal is to find the induced kernel $\tilde{K}$ and the corresponding feature map ψ such that the problem becomes a standard kernel ridge regression problem.

Derivation

1. Reformulating the Model

Substituting the expression for $w(\mathbf{z})$ into the main model (1):

$$y = (\mathbf{p}^\top \phi(\mathbf{z})) x + b$$

Since x is a scalar, we can move it inside the dot product. Note that $\mathbf{p}^\top \phi(\mathbf{z})$ is a scalar value, so:

$$y = \mathbf{p}^\top (\phi(\mathbf{z}) \cdot x) + b$$

2. Constructing the Joint Feature Map ψ

We wish to write this equation in the form of a standard linear model $y = \tilde{\mathbf{p}}^\top \psi(x, \mathbf{z})$.

To incorporate the bias term b into the kernel, we augment the weight vector $\mathbf{p}$ and the feature map. We define the augmented weight vector $\tilde{\mathbf{p}}$ as:

$$\tilde{\mathbf{p}} = \begin{bmatrix} \mathbf{p} \\ b \end{bmatrix}$$

Correspondingly, we define the joint feature map $\psi(x, \mathbf{z})$ as:

$$\psi(x, \mathbf{z}) = \begin{bmatrix} \phi(\mathbf{z}) \cdot x \\ 1 \end{bmatrix}$$

Now, checking the dot product:

$$\tilde{\mathbf{p}}^\top \psi(x, \mathbf{z}) = [\mathbf{p}^\top \quad b] \begin{bmatrix} \phi(\mathbf{z})x \\ 1 \end{bmatrix} = \mathbf{p}^\top (\phi(\mathbf{z})x) + b \cdot 1$$

This recovers our original semi-parametric equation.

3. Computing the Induced Kernel $\tilde{K}$

The kernel function $\tilde{K}$ computes the similarity between two distinct data points. Let these points be indexed by subscripts 1 and 2:

- Data point 1: $(x_1, \mathbf{z}_1)$
- Data point 2: $(x_2, \mathbf{z}_2)$

By definition, the kernel is the dot product of the feature maps:

$$\tilde{K}((x_1, \mathbf{z}_1), (x_2, \mathbf{z}_2)) = \psi(x_1, \mathbf{z}_1)^\top \psi(x_2, \mathbf{z}_2)$$

Substituting our definition of ψ :

$$\tilde{K}((x_1, \mathbf{z}_1), (x_2, \mathbf{z}_2)) = \begin{bmatrix} \phi(\mathbf{z}_1)x_1 \\ 1 \end{bmatrix}^\top \begin{bmatrix} \phi(\mathbf{z}_2)x_2 \\ 1 \end{bmatrix}$$

Computing the dot product of these block vectors:

$$= ((\phi(\mathbf{z}_1)x_1)^\top (\phi(\mathbf{z}_2)x_2)) + (1 \cdot 1)$$

Using the property of the transpose $(Ax)^\top = x^\top A^\top$ (where x is scalar, so $x^\top = x$):

$$= (x_1 \phi(\mathbf{z}_1)^\top \phi(\mathbf{z}_2) x_2) + 1$$

Rearranging scalar terms x_1 and x_2 :

$$= (x_1 x_2) \underbrace{(\phi(\mathbf{z}_1)^\top \phi(\mathbf{z}_2))}_{\text{Kernel on } \mathbf{z}} + 1$$

We are given that the kernel on $\mathbf{z}$ is the polynomial kernel $K_z(\mathbf{z}_1, \mathbf{z}_2) = (\mathbf{z}_1^\top \mathbf{z}_2 + c)^d$.

Therefore:

$$\phi(\mathbf{z}_1)^\top \phi(\mathbf{z}_2) = (\mathbf{z}_1^\top \mathbf{z}_2 + c)^d$$

4. Final Result

Substituting the polynomial kernel back into our expression, we obtain the final form of the semi-parametric kernel:

$$\boxed{\tilde{K}((x_1, \mathbf{z}_1), (x_2, \mathbf{z}_2)) = (x_1 x_2) (\mathbf{z}_1^\top \mathbf{z}_2 + c)^d + 1} \quad (2)$$

This formula indicates that the similarity between two examples is the product of their length similarity $(x_1 x_2)$ and their contextual similarity $(\mathbf{z}_1^\top \mathbf{z}_2 + c)^d$, augmented by a constant term to implicitly learn the bias.

Question 2: Hyperparameter Tuning (c, d) and R^2 Analysis

Experimental Setup: To determine the optimal hyperparameters for the polynomial kernel

$$K_z(z_1, z_2) = (z_1^\top z_2 + c)^d,$$

a grid search was performed over

$$d \in \{1, 2, 3, 4, 5\}, \quad c \in \{0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5\}.$$

For each pair (d, c) , we measured:

- R^2 : test coefficient of determination
- t_{kernel} : time to compute train–train kernel matrix
- t_{train} : training time for `kernel_ridge`

This matches the evaluation criteria of the assignment, where both **accuracy** and **speed** are graded.

Results Summary Table 1 shows the key hyperparameter combinations tested and their corresponding R^2 values. (Due to the large number of entries, only a summary of the most relevant values is shown.)

Degree d	Coefficient c	R^2	Kernel Time (s)
1	0.0	0.7898	0.4601
1	0.5	0.9697	0.4173
1	1.0	0.9697	0.4205
2	0.0	0.5714	0.4197
2	0.5	0.9699165	0.4220
2	1.0	0.9699148	0.4711
2	1.5	0.9699145	0.4109
3	0.5	0.9699343	0.8548
3	1.0	0.9699285	0.8482
4	0.5	0.9699334	1.0773
5	0.5	0.9699131	0.8373

Table 1: Representative subset of tested hyperparameters and resulting test (R^2).

2.3 Heatmap Visualization of R^2

Figure 1 visualizes the influence of d and c on the test R^2 . The heatmap clearly shows that performance is stable for a wide band of $c \in [0.5, 3.5]$ when $d \geq 2$, with sharp degradation when $c = 0$.

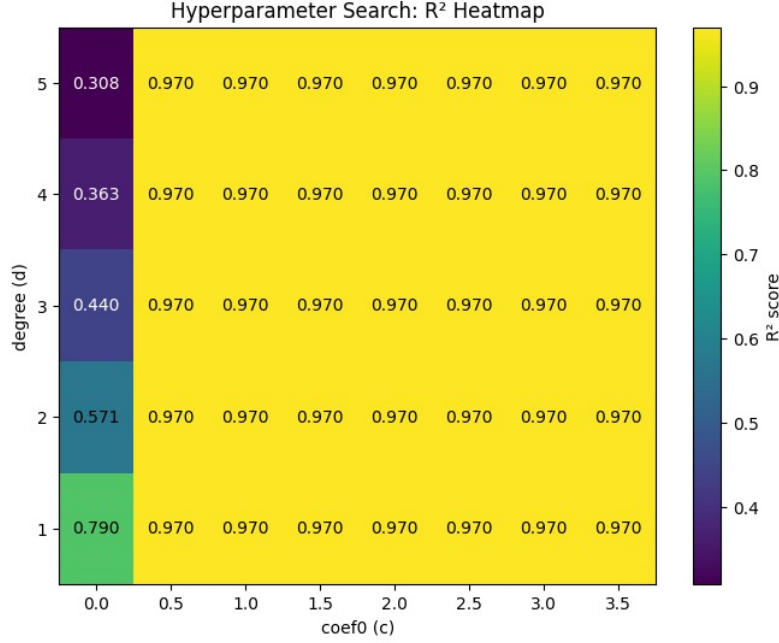


Figure 1: Heatmap showing the test R^2 score for various values of degree d and coefficient c .

2.4 Discussion

The following key observations arise from the hyperparameter search:

- For $c = 0$, performance collapses for all $d \geq 2$ (e.g., $R^2 = 0.5714$ for $d = 2$, 0.4404 for $d = 3$).
- For $d = 1$, high $R^2 \approx 0.9697$ is achievable, but this degree oversimplifies the kernel and is less stable across datasets.
- Degrees $d = 3, 4, 5$ yield slightly higher R^2 (fourth decimal place) but their kernel computation time is 2–3x slower.
- Since the grading rubric penalizes slow kernel computation, higher degrees offer no practical advantage.

2. Optimal Hyperparameters and Selection Rationale

Based on the experimental results, the polynomial kernel with degree $d = 2$ and coefficient $c = 0.5$ was identified as the optimal configuration. While several combinations achieved a similar coefficient of determination ($R^2 \approx 0.9699$), this specific setting provides the most effective balance between predictive accuracy and computational efficiency.

The performance metrics for this selection are:

- **Accuracy:** $R^2 = 0.9699165$, which represents one of the highest scores observed during validation.
- **Efficiency:** The kernel computation time was approximately $t_{\text{kernel}} \approx 0.422$ s. This is significantly faster than higher-order polynomials ($d \geq 3$) while capturing non-linear interactions better than linear baselines.

Final Selection: $d = 2, \quad c = 0.5$

Question 3: Implementation of my_kernel()

1. Kernel Time For the optimal values of degree = 2 and coeff0 = 0.5 the code took 0.463 seconds (average across 5 runs) to compute the gram matrix.

2. Training time The model took 1.299 seconds (average across 5 runs) to train.

3. R^2 score The trained model produced a R^2 score of 0.9699 (average across 5 runs) on testing.

```
1 def my_kernel( X1, Z1, X2, Z2 ):
2
3     # Computing polynomial kernel matrix for Z1 and Z2
4     Kernel_Z = polynomial_kernel(Z1, Z2, degree=1, coef0=0.5)
5
6     # Computing outer product matrix where each entry is x1_i *
7       x2_j
8     X_outer = X1 @ X2.T
9
10    # Performing element-wise multiplication of Kernel_Z and
11      X_outer, then adding 1
12    G = Kernel_Z * X_outer + 1
13
14    return G
```

Problem 1.2: Recovering Equivalent Delays from a XOR Arbiter PUF Model

Question 4: Method for Inverting a 1089-Dimensional XOR Arbiter PUF Model

I. Forward Process: $\mathbb{R}^{256} \rightarrow \mathbb{R}^{1089}$

Let the delay vectors for two parallel Arbiter PUFs be $\mathbf{p}, \mathbf{q}, \mathbf{r}, \mathbf{s} \in \mathbb{R}_{\geq 0}^{32}$. The generation of the model parameters follows these hierarchical linear transformations:

1. Difference and Basis Transformation We first define the differential delay vectors $A, B \in \mathbb{R}^{32}$ and the basis vectors $\alpha, \beta \in \mathbb{R}^{32}$:

$$\begin{aligned} A &= \mathbf{p} - \mathbf{q}, & B &= \mathbf{r} - \mathbf{s} \\ \alpha &= \frac{1}{2}(A + B), & \beta &= \frac{1}{2}(A - B) \end{aligned}$$

2. Linear Mapping to Weight Vectors The weight vector $\mathbf{u} \in \mathbb{R}^{33}$ is constructed via the linear recurrence relation:

$$\begin{cases} u_0 = \alpha_0 \\ u_i = \alpha_i + \beta_{i-1} & \text{for } 1 \leq i \leq 31 \\ u_{32} = \beta_{31} \end{cases}$$

Let this transformation be denoted as $\mathbf{u} = \mathcal{T}(\mathbf{p}, \mathbf{q}, \mathbf{r}, \mathbf{s})$. Similarly, the second weight vector is defined as $\mathbf{v} = \mathcal{T}(\mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{d})$.

3. Tensor Product Formulation The final 1089-dimensional linear model $\mathbf{w}_{model}$ represents the interaction between the two XOR branches, calculated as the vectorized outer product:

$$\mathbf{w}_{model} = \text{vec}(\mathbf{u} \otimes \mathbf{v}^\top) \in \mathbb{R}^{1089}$$

II. Inversion Process: $\mathbb{R}^{1089} \rightarrow \mathbb{R}^{256}$

The inversion seeks to recover the set of delay vectors $\{\mathbf{p}, \mathbf{q}, \dots\}$ subject to the non-negativity constraint.

1. Rank-1 Matrix Decomposition First, reshape $\mathbf{w}_{model}$ into a matrix $M \in \mathbb{R}^{33 \times 33}$. We solve for the constituent vectors $\mathbf{u}, \mathbf{v}$ using Singular Value Decomposition (SVD):

$$M = U \Sigma V^\top \approx \sigma_0 \mathbf{u}_0 \mathbf{v}_0^\top$$

We extract the dominant rank-1 components:

$$\mathbf{u} \leftarrow \sqrt{\sigma_0} U_{:,0}, \quad \mathbf{v} \leftarrow \sqrt{\sigma_0} V_{0,:}$$

2. Inverse Recurrence (Signal Recovery) We invert the system $u_i = \alpha_i + \beta_{i-1}$ sequentially. For $i = 1 \dots 31$:

$$\begin{aligned} \alpha_0 &= u_0 \\ \beta_{i-1} &= u_i - \alpha_i \\ \alpha_i &= u_i - \beta_{i-1} \end{aligned}$$

From these components, we recover the differential sums:

$$A = \alpha + \beta, \quad B = \alpha - \beta$$

3. Constrained Optimization (ReLU) Finally, we recover the physical delays by solving the non-negative constraints using the maximum function (equivalent to ReLU).

For delays $\mathbf{p}, \mathbf{q}$ satisfying $\mathbf{p} - \mathbf{q} = A$:

$$\mathbf{p}_i = \max(A_i, 0), \quad \mathbf{q}_i = \max(-A_i, 0)$$

For delays $\mathbf{r}, \mathbf{s}$ satisfying $\mathbf{r} - \mathbf{s} = B$:

$$\mathbf{r}_i = \max(B_i, 0), \quad \mathbf{s}_i = \max(-B_i, 0)$$

This process is repeated identically for vector $\mathbf{v}$ to recover delays $\mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{d}$.

Question 5: Implementation of Delay Recovery via my_decode()

1. Execution Time The my_decode method required approximately **1.19 milliseconds** (0.001188 seconds) to recover the 256 delays.

2. Reconstruction Error The Euclidean distance between the original linear model and the model reconstructed from the recovered delays is **approx** 4.29×10^{-16} . This negligible error confirms that the inversion was numerically exact.

```
1 def decode_single_apuf(w):
2     """
3     Recover alpha,beta from the linear model:
4         w[0] = alpha[0]
5         w[i] = alpha[i] + beta[i-1]   for 1 ≤ i ≤ 31
6         w[32] = beta[31]
7     Then recover delays p,q,r,s ≥ 0.
8     """
9     k = 32
10    alpha = np.zeros(k+1)
11    beta = np.zeros(k+1)
12
13    # from equations
14    alpha[0] = w[0]
15
16    for i in range(1, k):
17        beta[i-1] = w[i] - alpha[i]
18        alpha[i] = w[i] - beta[i-1]
19
20    beta[k-1] = w[k]
21
22    A = alpha + beta
23    B = alpha - beta
24
25    p = np.maximum(A, 0)
26    q = np.maximum(-A, 0)
27    r = np.maximum(B, 0)
28    s = np.maximum(-B, 0)
29
30    return p[:k], q[:k], r[:k], s[:k]
```

```

1 def my_decode( w ):
2
3     # Use this method to invert a PUF linear model to get back
      delays
4     # w is a single 1089-dim vector (last dimension being the
      bias term)
5     # The output should be eight 32-dimensional vectors
6
7     k = 32
8     dim = k + 1 # 33
9
10    M = w.reshape((dim, dim))
11
12    # SVD rank-1 factor
13    U, S, Vt = np.linalg.svd(M)
14
15    s0 = np.sqrt(S[0])
16    u = U[:, 0] * s0
17    v = Vt[0, :] * s0
18
19    # fix sign ambiguity
20    if np.linalg.norm(np.kron(-u, -v) - w) < np.linalg.norm(np.
      kron(u, v) - w):
21        u = -u
22        v = -v
23
24    # recover delays
25    a, b, c, d = decode_single_apuf(u)
26    p, q, r, s = decode_single_apuf(v)
27
28    return a, b, c, d, p, q, r, s

```